SISTEMAS OPERATIVOS EMBEBIDOS

ARQUITECTURA, FUNCIONAMIENTO Y APLICACIONES

Realizado por:
Casas Pérez Rodrigo Imanol
Malfavon Lemus Paris Samantha

¿QUE ES
UN SISTEMA EMBEBIDO?



Un sistema embebido es un sistema de cómputo integrado en un producto para cumplir funciones específicas. Combina hardware y software, y suele operar con restricciones de costo, energía, tamaño o tiempo de respuesta. No todos requieren sistema operativo: algunos ejecutan firmware directamente sobre el hardware (bare-metal).



Los sistemas operativos no se encuentran únicamente en computadoras personales. Muchos dispositivos dedicados, como routers, televisores inteligentes y controladores industriales, necesitan administrar hardware, ejecutar tareas y comunicarse con otros equipos

CARACTERÍSTICAS Y LIMITACIONES

1. Recursos ajustados: memoria, almacenamiento y capacidad de cómputo según la aplicación.
2. Eficiencia energética: importante en dispositivos alimentados por batería.
3. Confiabilidad: operación estable y recuperación ante fallos.
4. Especialización: solo se integran los servicios necesarios.
5. Tiempo real: algunos sistemas deben cumplir plazos de respuesta verificables.



ARQUITECTURA DE UN SO EMBEBIDO

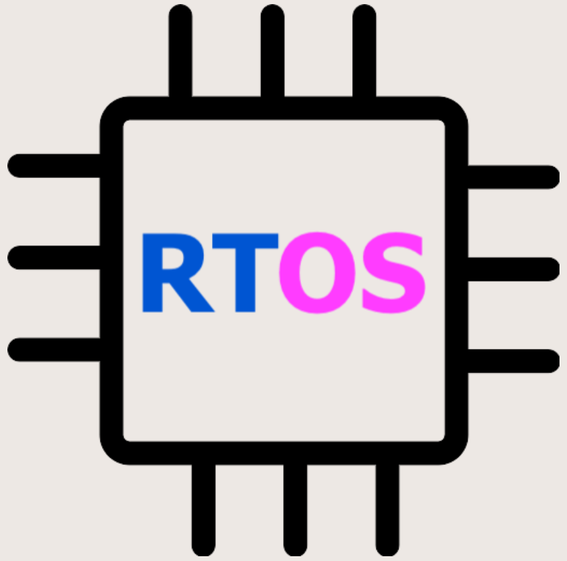
Las aplicaciones normalmente trabajan en espacio de usuario y solicitan servicios al kernel mediante llamadas al sistema. Los controladores suelen formar parte del kernel o cargarse como módulos.

- Aplicaciones: realizan la función del dispositivo.
- Bibliotecas y servicios: ofrecen interfaces y funciones reutilizables.
- Kernel: administra tareas, memoria, interrupciones y acceso al hardware.
- Controladores: permiten comunicarse con periféricos.
- Hardware: procesador, RAM, almacenamiento y dispositivos de entrada/salida.

TIPOS DE SO EMBEBIDOS

- RTOS: priorizan un comportamiento temporal predecible; ejemplos FreeRTOS y Zephyr.
- Linux embebido: sistemas basados en el kernel Linux, personalizados para dispositivos con necesidades de multitarea, conectividad y servicios.

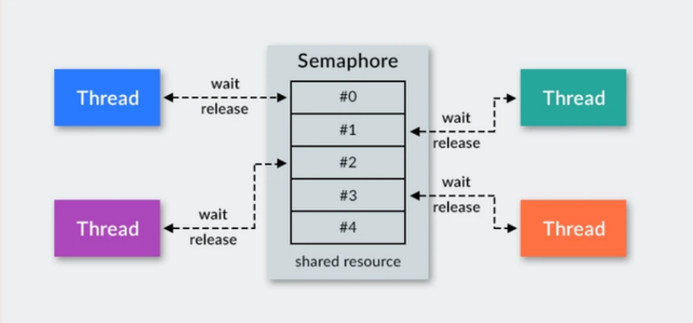
La elección depende de memoria, hardware, funciones y requisitos de respuesta.

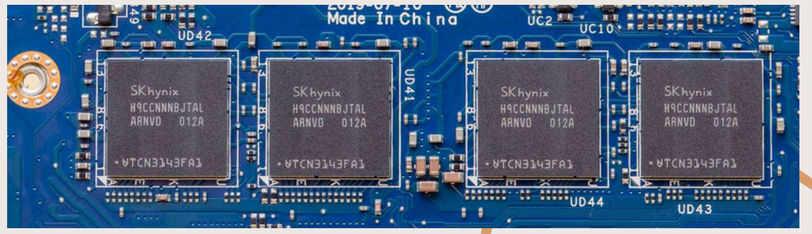




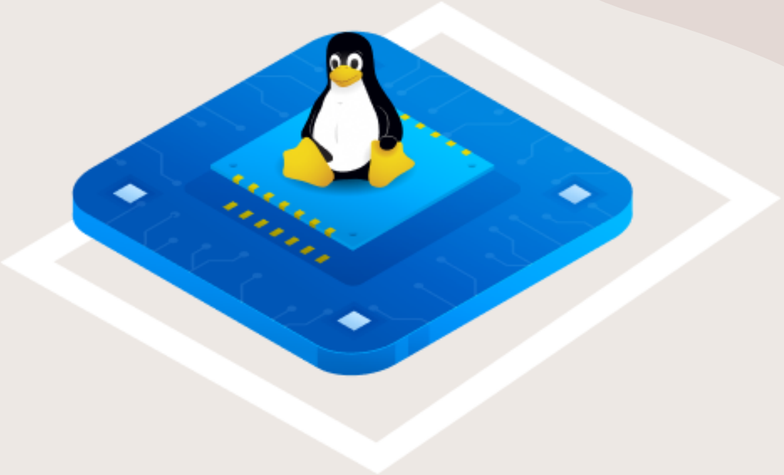
PROCESOS, PLANIFICACIÓN Y MEMORIA

- Planificación: el kernel decide qué tarea o hilo utiliza la CPU.
- Prioridades: en planificadores expulsivos, una tarea lista de mayor prioridad puede desplazar a otra. Interrupciones: permiten atender eventos del hardware.
- Memoria: se administran pilas, buffers y asignaciones según los recursos disponibles.
- Sincronización: semáforos, mutex y colas coordinan tareas concurrentes.

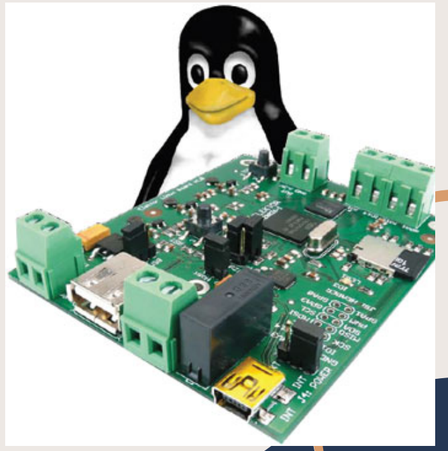




LINUX EN SISTEMAS EMBEBIDOS

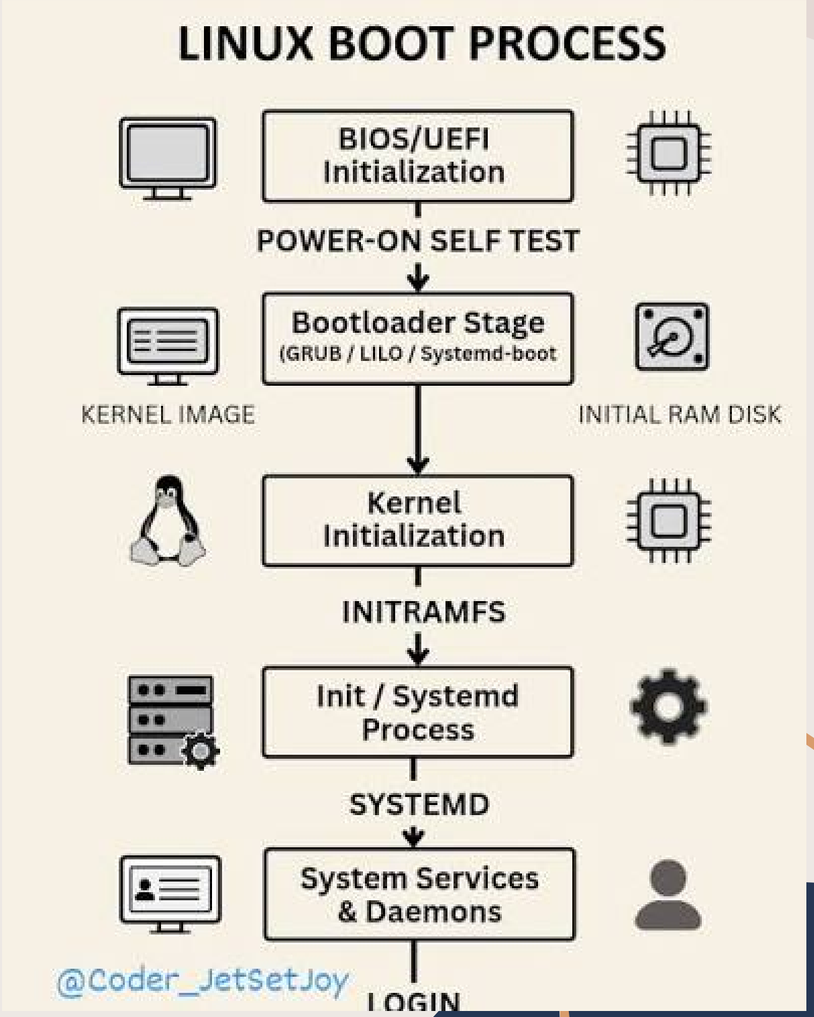


Linux embebido utiliza el kernel Linux dentro de un sistema adaptado a un dispositivo concreto.
Una imagen puede contener bootloader, kernel, sistema de archivos raíz, utilidades y aplicación principal.
Buildroot automatiza la generación de imágenes Linux para equipos embebidos.
Yocto Project proporciona herramientas y procesos para construir sistemas Linux personalizados y mantenibles



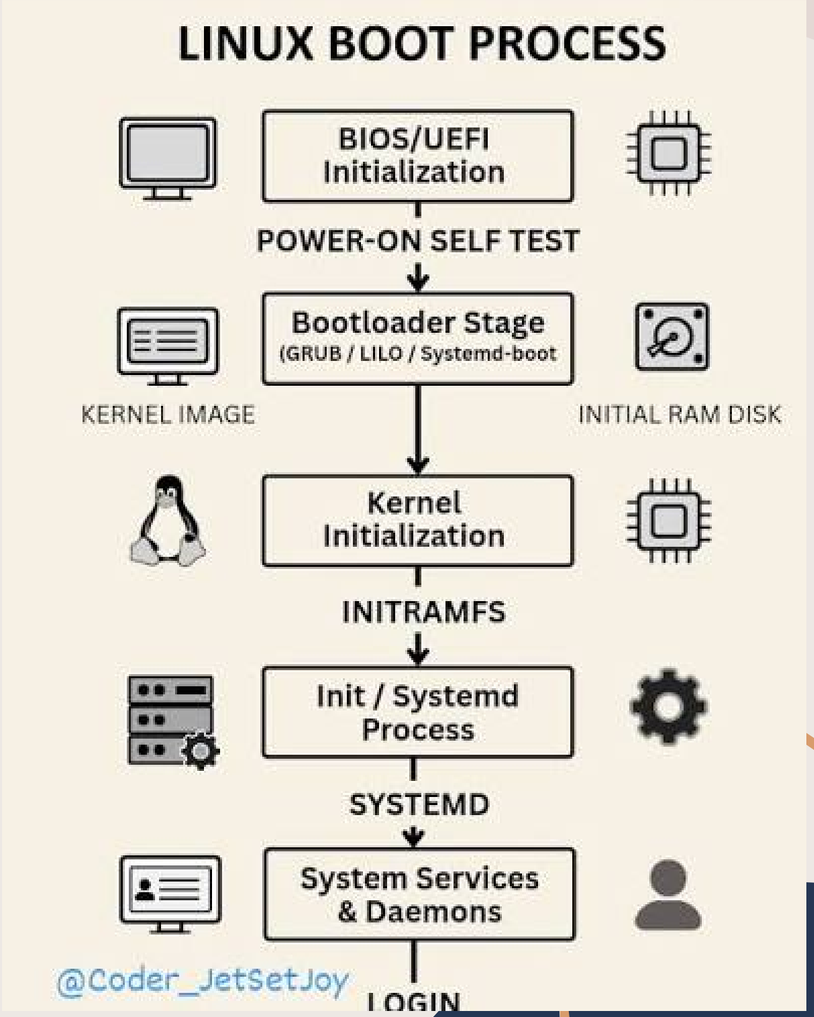
BOOTLOADER

Un bootloader (o gestor de arranque) es el programa que se ejecuta tras encender la computadora (desde la BIOS o UEFI) y cuya función principal es cargar el núcleo (kernel) del sistema operativo Linux en la memoria RAM



BOOTLOADER

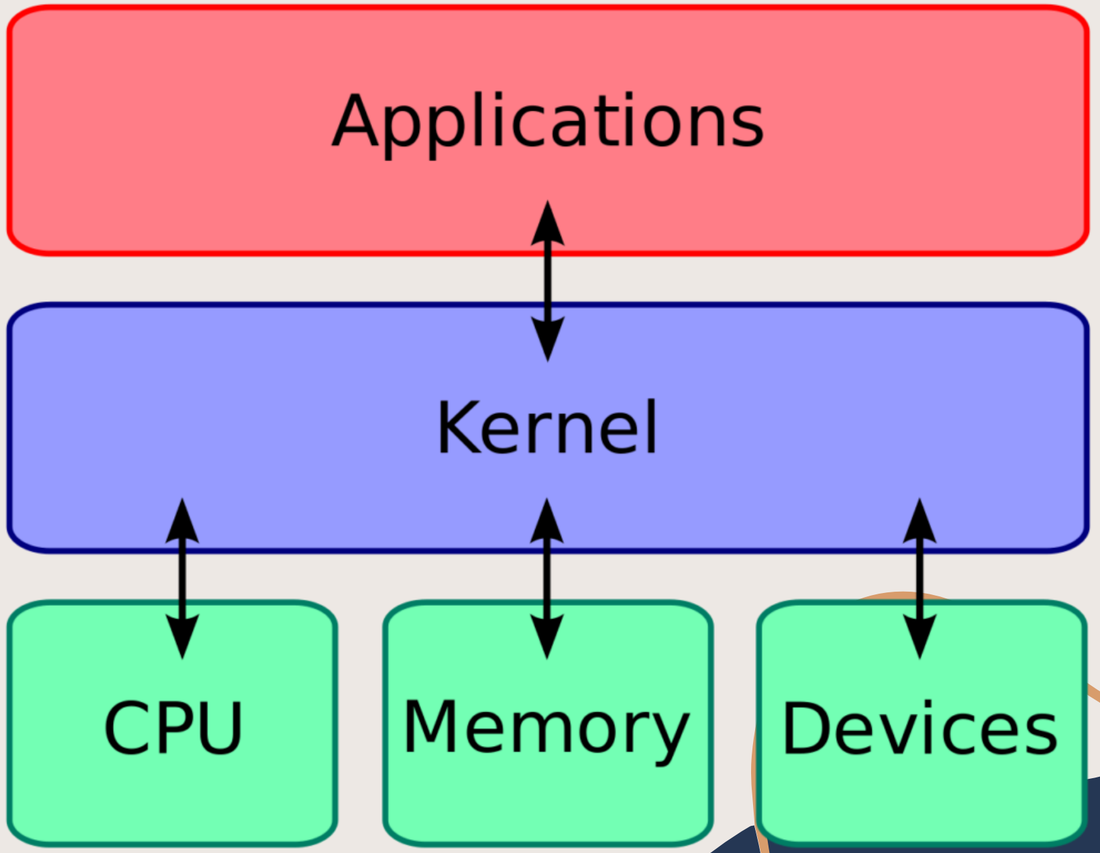
Un bootloader (o gestor de arranque) es el programa que se ejecuta tras encender la computadora (desde la BIOS o UEFI) y cuya función principal es cargar el núcleo (kernel) del sistema operativo Linux en la memoria RAM



KERNEL

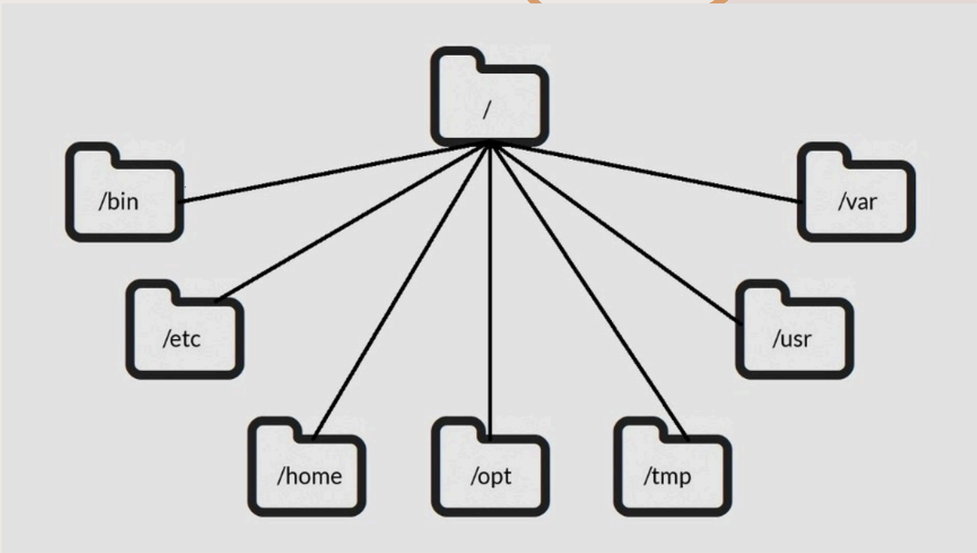
El kernel de Linux es el núcleo o corazón del sistema operativo que actúa como un puente directo entre el hardware de la computadora y los programas de usuario

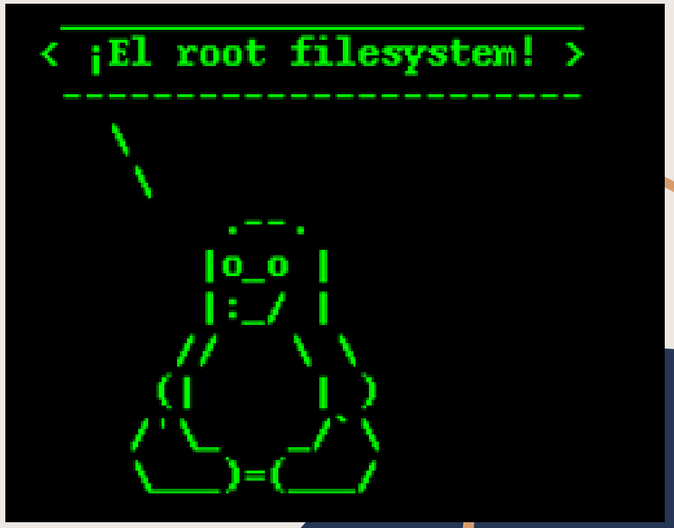
- Gestión de la CPU:
- Gestión de memoria
- Controladores de dispositivos (drivers):
- Pila de red:
- Llamadas al sistema



ROOT FILESYSTEM

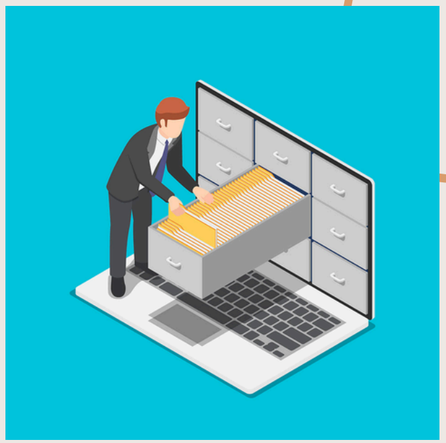
El sistema de archivos raíz (root filesystem o rootfs) es la base primordial del árbol jerárquico de directorios en un sistema operativo Linux. Se representa universalmente con una barra diagonal (/) y contiene los componentes, utilidades de configuración, librerías y binarios esenciales para iniciar el sistema, montar otros sistemas de archivos y permitir reparaciones de emergencia.



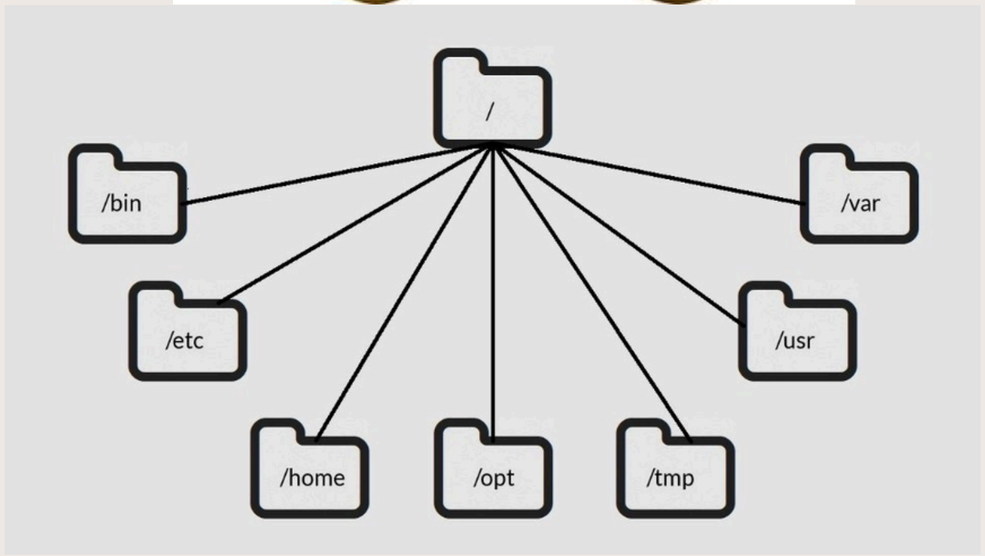



JAVA
C++
CONNECTED








/
/bin
/var
/etc
/usr
/home
/opt
/tmp





root@vps:~

```
top - 15:54:04 up 4 days, 16:52,  1 user,  load average: 0,89, 0,70, 0,55
Tasks:  61 total,   1 running,  60 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0,0 us,  0,0 sy,  0,0 ni,100,0 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
MiB Mem :   4096,0 total,    768,8 free,   1212,0 used,   2115,2 buff/cache
MiB Swap:      0,0 total,      0,0 free,      0,0 used.   2529,2 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0  240564   9600   7680 S   0,0   0,2  12:45.84 systemd
    695 root      20   0  153980  51648  47168 S   0,0   1,2   1:01.93 systemd+
    709 root      20   0   81664   7040   6400 S   0,0   0,2   0:12.67 systemd+
    710 dbus      20   0   54732   3840   3840 S   0,0   0,1   1:26.65 dbus-da+
    770 root      20   0  376444   9004   5120 S   0,0   0,2   0:12.68 raiolac+
    771 admin     20   0  388408  14784  10880 S   0,0   0,4   0:00.07 raiolac+
    772 admin     20   0  388408  14144   9600 S   0,0   0,3   0:00.06 raiolac+
    984 root      20   0  639940  38400  31360 S   0,0   0,9   0:21.82 php-fpm
    991 root      20   0   76728   5760   5760 S   0,0   0,1   0:21.25 sshd
        root      20   0  339272  29564  28924 S   0,0   0,7   0:49.36 rsyslogd
        exim      20   0  109252  12160   9600 S   0,0   0,3   0:28.52 exim
        root      20   0   27328    640    640 S   0,0
        root      20   0  219332   1920   1920 S   0,0
        root      20   0  219332   1920   1920 S   0,0
        root      20   0  219332   1920   1920 S   0,0
        root      20   0   26840   3840   3840 S   0,0
        named     20   0  869380  78720   7040 S   0,0
```

Wi-Fi Networks
Select a network

- Office TV.m
- eduroam
- ITS-IPv6-TEST
- PoC-Test
- MGuest
- MWireless
- MSetup
- TEST-PacketFence

2

3

Cancel

Connect

Bluetooth Devices

Adapter Device View Help

Search Setup... Send File

(AD)Logitech Adapter
Uncategorized
C8:84:47:02:43:AD

S68H
Cordless
00:01:E3:FD:2E:A3

Nokia N8
Smart phone
9C:4A:7B:F0:AF:16

40.98 MB *41.09 KB/s* 233.01 KB *198.50 B/s*

APLICACION PRINCIPAL

La aplicación principal es el programa encargado de realizar las funciones específicas para las que fue diseñado el dispositivo.

Por ejemplo, en un router, esta aplicación puede encargarse de administrar las conexiones de red y permitir que los dispositivos se comuniquen entre sí.

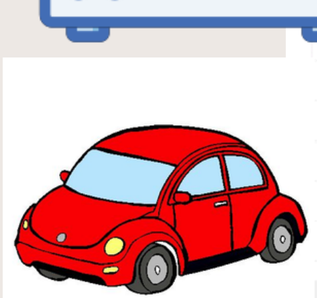
En pocas palabras, es el software que cumple el propósito principal del dispositivo, utilizando los recursos y servicios que proporciona el sistema operativo.



APLICACIONES EN EL MUNDO REAL

- Routers: servicios de red, firewall y administración remota.
- Sistemas multimedia: reproducción, interfaces y conectividad.
- Automatización industrial: supervisión, comunicación y control.
- Automóviles: sistemas de información y entretenimiento, además de unidades de control especializadas.
- Dispositivos IoT: sensores, gateways y equipos conectados.









VENTAJAS, DESAFÍOS Y SEGURIDAD

01

Ventajas:

especialización, automatización, reutilización de software y conectividad.

02

Desafíos:

Límites de recursos, compatibilidad de controladores, mantenimiento y consumo energético.

03

Seguridad:

Proteger el arranque, limitar servicios expuestos, corregir vulnerabilidades y distribuir actualizaciones verificadas.

CONCLUSION

En conclusión, los sistemas operativos embebidos son fundamentales en la tecnología actual, ya que permiten administrar los recursos y ejecutar tareas específicas de manera eficiente.

Como vimos, existen diferentes tipos, como los RTOS y Linux embebido, además de herramientas como Buildroot y Yocto Project, que facilitan su desarrollo y personalización.

Aunque presentan desafíos relacionados con los recursos, el mantenimiento y la seguridad, su importancia sigue creciendo debido a sus múltiples aplicaciones en la industria y en nuestra vida cotidiana.

BIBLIOGRAFÍA

Amazon Web Services. (s. f.). RTOS fundamentals. FreeRTOS. https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/01-RTOS-fundamentals

Zephyr Project. (s. f.). Zephyr Project documentation. https://docs.zephyrproject.org/latest/
Buildroot Developers. (s. f.). The Buildroot user manual.

https://buildroot.org/downloads/manual/manual.html
Yocto Project. (s. f.). Overview and concepts manual. https://docs.yoctoproject.org/overview-manual/

The Linux Kernel Development Community. (s. f.). The Linux kernel documentation.
https://docs.kernel.org/